

create proper document for

MongoDB CRUD Operations Using FastAPI and Django

Introduction

This project demonstrates CRUD (Create, Read, Update, Delete) operations using two Python frameworks:

1. **FastAPI + MongoDB**
2. **Django + MongoDB**

The objective of this project is to understand how different Python frameworks interact with MongoDB and how CRUD operations are implemented in both API-first and web application frameworks.

Project Objective

The main goals of this project are:

- Learn MongoDB integration with Python.
 - Understand REST API development.
 - Implement CRUD operations.
 - Compare FastAPI and Django.
 - Understand API testing using Swagger and Postman.
-

Technologies Used

Backend Frameworks

- FastAPI
- Django

Database

- MongoDB Community Server

Libraries

Python ▾

```
1 fastapi
2 uvicorn
3 pymongo
4 python-dotenv
5 django
6
```

Tools

- MongoDB Compass
- Postman
- VS Code
- Git
- GitHub

What is MongoDB?

MongoDB is a NoSQL database that stores data in JSON-like documents.

Example:

JSON ▾

```
1 {
2   "student_id": 1,
3   "name": "Vijay",
4   "age": 22
5 }
6
```

Unlike SQL databases, MongoDB stores data in collections instead of tables.

Project Architecture

[Plain Text](#) ▾

```
1 MongoDB Server
2 |
3 |— fastapi_student_db
4 |   |— students
5 |
6 |— django_student_db
7 |   |— students
8
```

Both applications use the same MongoDB server but different databases.

FastAPI + MongoDB

Why FastAPI?

FastAPI is designed specifically for building APIs.

Features:

- High Performance
- Automatic Swagger Documentation
- Automatic ReDoc Documentation
- Data Validation
- Easy Development

FastAPI Project Structure

[Plain Text](#) ▾

```
1 fastapi_mongodb
2 |
3 |— app
4 |   |— main.py
5 |   |— database.py
6 |   |— models.py
```

```
7 |  
8 |└─ .env  
9 |└─ requirements.txt  
10
```

Database Connection

Python ▾

```
1 client = MongoClient("mongodb://localhost:27017")  
2 db = client["fastapi_student_db"]  
3 student_collection = db["students"]  
4  
5
```

```
student_collection = db["students"]
```

Student Model

Python ▾

```
1 class Student(BaseModel):  
2     student_id: int  
3     name: str  
4     age: int  
5
```

FastAPI CRUD Operations

Create Student

HTTP ▾

```
1 POST /students  
2
```

Request:

JSON ▾

```
1 {  
2   "student_id": 1,  
3   "name": "Vijay",  
4   "age": 22  
5 }  
6
```

Get All Students

HTTP ▾

```
1 GET /students  
2
```

Get Student By ID

HTTP ▾

```
1 GET /students/1  
2
```

Update Student

HTTP ▾

```
1 PUT /students/1  
2
```

Delete Student

HTTP ▾

```
1 DELETE /students/1
2
```

FastAPI Documentation

Swagger UI:

Plain Text ▾

```
1 http://127.0.0.1:8000/docs
2
```

ReDoc:

Plain Text ▾

```
1 http://127.0.0.1:8000/redoc
2
```

Django + MongoDB

Why Django?

Django is a full-stack web framework.

Features:

- Rapid Development
 - Security Features
 - Authentication
 - Admin Panel
 - Scalable Architecture
-

Django Project Structure

Plain Text ▾

```
1  django_mongodb
2  |
3  ├── config
4  |
5  ├── studentcrud
6  |   ├── views.py
7  |   ├── urls.py
8  |   └── mongodb.py
9  |
10 ├── manage.py
11 └── db.sqlite3
12
```

MongoDB Connection

Python ▾

```
1  client = MongoClient(
2  "mongodb://localhost:27017"
3  )
4  db = client["django_student_db"]
5  student_collection = db["students"]
6
7
```

```
student_collection = db["students"]
```

Django CRUD Operations

Create Student

HTTP ▾

```
1 POST /students/  
2
```

Request Body:

JSON ▾

```
1 {  
2   "student_id": 1,  
3   "name": "Vijay",  
4   "age": 22  
5 }  
6
```

Get All Students

HTTP ▾

```
1 GET /students/  
2
```

Get Student By ID

HTTP ▾

```
1 GET /students/1/  
2
```

Update Student

HTTP ▾

```
1 PUT /students/1/  
2
```

Request:

JSON ▾

```
1 {  
2   "student_id": 1,  
3   "name": "Vijay Kumar",  
4   "age": 23  
5 }  
6
```

Delete Student

HTTP ▾

```
1 DELETE /students/1/  
2
```

API Testing

FastAPI was tested using:

Plain Text ▾

```
1 Swagger UI  
2
```

Django was tested using:

Plain Text ▾

```
1 Postman
2
```

CRUD Flow

Create

Plain Text ▾

```
1 Client
2 ↓
3 FastAPI / Django
4 ↓
5 MongoDB
6
```

Read

Plain Text ▾

```
1 Client
2 ↓
3 Framework
4 ↓
5 MongoDB
6 ↓
7 Response
8
```

Update

Plain Text ▾

```
1 Client
2 ↓
3 Framework
4 ↓
5 MongoDB Update
6
```

Delete

Plain Text ▾

```
1 Client
2 ↓
3 Framework
4 ↓
5 MongoDB Delete
6
```

⚠ Challenges Faced

1. MongoDB Database Not Visible

Cause:

Plain Text ▾

```
1 Database had no documents.
2
```

Solution:

Plain Text ▾

```
1 Insert first document.
2
```

2. CSRF Error in Django

Error:

```
1 403 Forbidden
2
```

Plain Text ▾

Solution:

```
1 @csrf_exempt
2
```

Python ▾

used for API requests.

3. URL Routing Errors

Error:

```
1 path is not defined
2
```

Plain Text ▾

Solution:

```
1 from django.urls import path
2
```

Python ▾

FastAPI vs Django

	≡ Feature	≡ FastAPI	≡ Django
1	API Development	✓ Excellent	✓ Good
2	Swagger UI	✓ Built-in	✗ Requires Additional Setup
3	Performance	✓ High	✓ Good
4	Web Applications	✗ Limited	✓ Excellent
5	Learning Curve	✓ Easy	✓ Moderate

Learning Outcomes

Through this project I learned:

- MongoDB CRUD operations.
- FastAPI development.
- Django development.
- REST API design.
- MongoDB Compass usage.
- Swagger API testing.
- Postman API testing.
- Git and GitHub project management.

Conclusion

This project successfully demonstrates CRUD operations using both FastAPI and Django with MongoDB.

The FastAPI application focuses on API development with built-in documentation, while the Django application demonstrates CRUD operations using URL-based API endpoints and MongoDB integration.

Both applications interact with separate MongoDB databases and perform Create, Read, Update, and Delete operations successfully.
